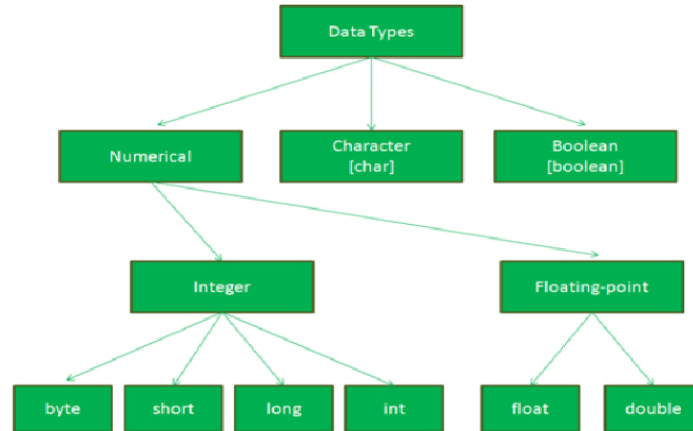


Object Oriented Programming In Java

Saurabh Pandey
Associate Professor
Sitare University

Types of Variables : Primitive

1. Primitive
2. Reference or object variable



Types of Variables : Primitive

Primitive:

*Java thinks anything with a floating point is a double, unless you use 'f'.

*Floats can represent decimal values up to 7 digits of precision, and 5 digits of precision.

Type	Size (in bits)	Range
byte	8	-128 to 127
short	16	-32,768 to 32,767
int	32	-2^{31} to $2^{31}-1$
long	64	-2^{63} to $2^{63}-1$
float	32	1.4e-045 to 3.4e+038
double	64	4.9e-324 to 1.8e+308
char	16	0 to 65,535
boolean	1	true or false

Types of Variables : Primitive

Variable Names:

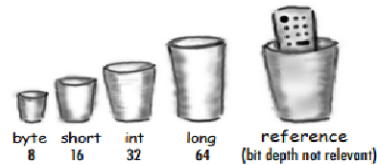
you need a name and a type for your variables.

- It must start with a letter, underscore (_), or dollar sign (\$). You can't start a name with a number.
- After the first character, you can use numbers as well. Just don't start it with a number.
- It can be anything you like, subject to those two rules, just so long as it isn't one of Java's reserve words.

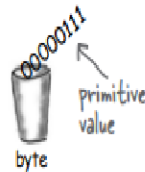
Types of Variables : Reference or Object variable

Object Variable:

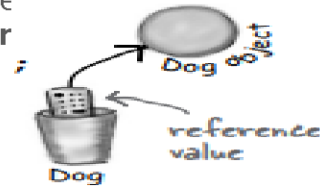
- There is actually no such thing as an object variable.
- There's only an object reference variable.
- An object reference variable holds bits that represent a way to access an object.
- It doesn't hold the object itself, but it holds something like a pointer. Or an address. Except, in Java we don't really know what is inside a reference variable. We do know that whatever it is, it represents one and only one object. And the JVM knows how to use the reference to



lwa



d in Gar



Life and death on garbage collectible heap

```
Book b = new Book();
```

```
Book c = new Book();
```

```
Book d = c;
```

```
c = b;
```

```
D = null;
```

Show pictorial presentation.

Encapsulation

Encapsulation is the process that binds together the data and code into a single unit and keeps both from being safe from outside interference and misuse. In this process, the data is hidden from other classes and can be accessed only through the current class's methods. Hence, it is also known as **data hiding**.

To achieve encapsulation in java, follow certain steps as proposed below:

- Declare the variables as private
- Declare the setters and getters to set and get the variable values

There are few advantages of encapsulation in java as follows:

- **Control Over Data:** We can write the logic in the setter method to not store the negative values for an Integer. So, by this way we can control the data.
- **Data Hiding:** The data members are private so other class can't access the data members.

Example:

```
// A Java class which is a fully encapsulated class.
public class Car {

    // private variable
    private String name;

    // getter method for name
    public String getName(){
        return name;
    }

    // setter method for name
    public void setName(String name){
        this.name = name;
    }
}

// Java class to test the encapsulated class.
public class Test {
    public static void main(String[] args)
    {

        // creating instance of the encapsulated class
        Car car
        = new Car();

        // setting value in the name member
        car.setName("Honda");

        // getting value of the name member
        System.out.println(car.getName());

    }
}
```

Variable default values

Instance variables always get a default value. If you don't explicitly assign a value to an instance variable, or you don't call a setter method, the instance variable still has a value!

integers 0

floating points 0.0

boolean false

references null

Local variables do NOT get a default value! The compiler complains if you try to use a local variable before the variable is initialized.